SYSTEMS INTEGRATION · LAB 1 · SIMPLE WEB SHOP

Lab 1 Guide
VeloCity Bike Shop

What did Lab 1 ask for, what did we build and how, and where is it in the code? Architecture, models, data model, SAP ERP integration, SAP connectors, cloud deployment and questions that may come up in the presentation.

Sources: `lab/simple-web-shop.md` in the lecture repository and our project folder `Desktop\webshop` (every line of the code has an English comment).

Prerequisite: the Lab 0 guide (Mini-ERP, SAP BTP, HANA). This guide builds on it.

Live addresses: local `http://localhost:5173` · cloud `https://369eca91trial-dev-velocity-webshop.cfapps.us10-001.hana.ondemand.com`

Contents

1. What does Lab 1 ask for? . . . 3

2. Short answer: where is each requirement met? . . . 3

3. Architecture and the models we used . . . 4

Technology stack . . . 5

4. Project structure: what is where? . . . 5

5. Data model (SQLite) . . . 6

6. Product management (R1) . . . 7

7. Customer management (R2) . . . 8

8. Order flow (R3): “ERP first” . . . 8

Status changes . . . 9

9. SAP ERP integration and SAP connectors . . . 9

SAP connectors (Admin › SAP connectors) . . . 9

10. Running it: local and cloud . . . 10

Local . . . 10

Cloud (SAP BTP Cloud Foundry) . . . 10

11. Design decisions (to defend) . . . 11

12. Questions the lecturer may ask . . . 12

13. Known limitations . . . 13

14. Glossary (short) . . . 13

1. What does Lab 1 ask for?

Task: design and develop a simple web shop that manages products, customers and orders. The technology is free to choose; the interface may be simple, but the basic features must work. The last sentence matters: **“Prepare to discuss your design decisions.”** – we must be able to defend our design decisions (sections 11–12).

No	Requirement (from the lab text)
R1.1	**Product Management:** show a list of the available products.
R1.2	New products can be added and existing products removed.
R1.3	Every product has at least a **name, description and price**.
R2.1	**Customer Management:** customer accounts – register, log in, update personal information.
R2.2	Every customer has at least a **name, email, address and purchase history**.
R3.1	**Order Processing:** customers add products to an order and create a new order with “checkout”.
R3.2	Every order stores the **customer, products, quantities, total price** and a **status** (e.g. pending, shipped, completed).
Extra	Free design (stack, data storage, structure). Simplicity and functionality matter. Design decisions are discussed.

2. Short answer: where is each requirement met?

No	How did we meet it?	Screen (client)	API / code (server)
R1.1	Product list with categories, search, sorting and stock badges; product detail page.	`pages/ShopPage.jsx` (/), `ProductPage.jsx`	`GET /api/products`, `routes/products.js`
R1.2	Admin › Products: add, edit, remove (soft delete) and restore. With an ERP connected, the ERP is written first.	`pages/AdminPage.jsx` (ProductForm)	`POST/PUT/DELETE /api/products`, `services/erpMasterData.js`
R1.3	name, description, price + productID (ERP key), currency, stock, category.	—	`products` table (`db.js`)
R2.1	Register, log in/out, update profile + address + phone, change password.	`RegisterPage.jsx`, `LoginPage.jsx`, `AccountPage.jsx`	`/api/auth/register|login|logout`, `GET/PUT /api/me`, `auth.js`
R2.2	name, email, address (street, number, postal code, city, country) + phone; purchase history in “My account › Orders”.	`AccountPage.jsx`	`customers` table, `GET /api/orders`
R3.1	Cart → Checkout (address + summary) → confirmation page. The order is created in the SAP ERP first.	`CartPage.jsx`, `CheckoutPage.jsx`, `ConfirmationPage.jsx`	`POST /api/orders`, `services/erpOrders.js`
R3.2	orders + order_items: customer, products (with a price snapshot), quantities, total, status NEW → PICKED → SHIPPED → COMPLETED / CANCELED.	`OrderCard.jsx`, Admin › Orders	`orders`, `order_items`, `PATCH /api/orders/:id/status`

What we did beyond the lab requirements

- The shop is integrated with the **SAP Mini-ERP** of Lab 0: products, prices and stock come from the ERP, orders are created in the ERP and the stock goes down there.
- **SAP connectors**: switch between the local ERP and the ERP on BTP with one click.
- The shop itself is deployed to SAP BTP Cloud Foundry too (local + cloud at the same time).
- Admin panel (dashboard, products, orders, customers), demo accounts, responsive interface.
- Lab 2 additions (file transfer, supplier portal + FTP) are described in README.md.

3. Architecture and the models we used

The answer to “which model did you use?” has four parts: the **architecture model**, the **data model**, the **integration model** and the **security model**.

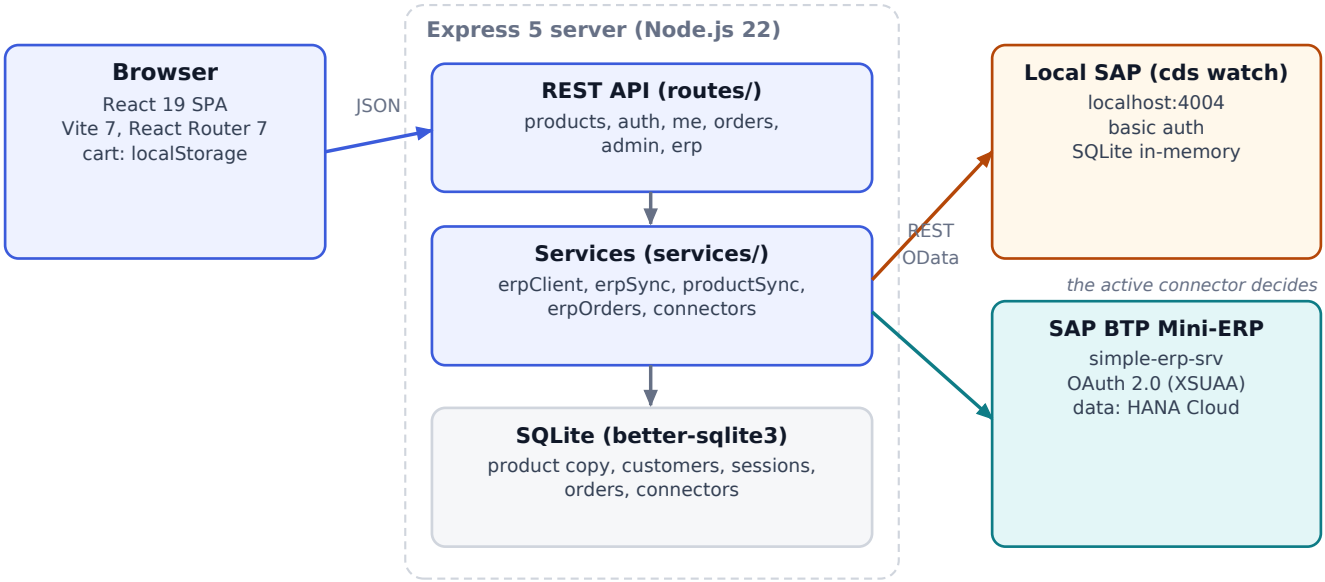


Figure 1 – Architecture: the browser only talks to our own API; only the server (SAP connector) talks to the ERP.

Model	What we used	Why?
Architecture model	**Client–server, 3 tiers**: presentation (React SPA) → application (Express REST API) → data (SQLite) + a separate **integration layer** (ERP client, SAP connectors).	Separate responsibilities; the interface does not know the ERP, and ERP secrets never reach the browser.
Data model	**Relational**: products, customers, sessions, orders, order_items, erp_connectors, settings (section 5).	Product–customer–order relations fit tables naturally; very close to the ERP’s model.
Integration model	**ERP = master (system of record)**; the shop keeps a local copy. Orders: synchronous request/response (OData POST, “ERP first”). Products and order status: **periodic pull** (60 s) + synchronization on events.	Stock and orders have one source of truth; the shop is fast and can show the last known data even when the ERP is down for a short time.
Security model	Shop users: bcrypt passwords + server-side session tokens, roles customer/admin. System to system: basic auth locally, **OAuth 2.0 client credentials** (XSUAA) on BTP.	Human and machine identities are separate; ERP secrets stay on the server.

Technology stack

Layer	Technology	Note
Frontend	React 19, React Router 7, Vite 7, plain CSS	No UI library; our own design system (styles.css).
Backend	Node.js 22, Express 5	Async errors go to the central error handler.
Database	SQLite (better-sqlite3 12)	No installation; synchronous and fast; transactions.
Security	bcryptjs, random session tokens (24 hours)	Sessions live on the server; deleted at logout.
ERP	SAP CAP Mini-ERP: REST `/rest/api` + OData v4 `/odata/v4/simple-erp`	Local: basic auth (service-user). BTP: XSUAA token.
Running	npm workspaces + concurrently; `start-local.cmd`	Server + client with one command.
Cloud	SAP BTP Cloud Foundry, nodejs_buildpack, `manifest.yml` (created by cf-prepare.mjs)	ERP credentials through a service binding.

4. Project structure: what is where?

```
webshop/
├─ start-local.cmd            local demo: ERP (cds watch) + shop with one double-click
├─ package.json               npm workspaces; "npm run dev", "npm run cf:build"
├─ scripts/cf-prepare.mjs     cloud package: creates cf-dist/ + manifest.yml
├─ server/
│  ├─ .env.example            settings template (.env and xsuaa-key.json are NEVER shared)
│  └─ src/
│     ├─ index.js             Express app, routes, error handling, ERP sync at start
│     ├─ db.js                SQLite schema, schema upgrades, demo data, connectors
│     ├─ auth.js              password hashes, sessions, requireAuth / requireAdmin
│     ├─ catalog.js           demo catalog with 14 products, categories
│     ├─ routes/              products, customers (auth + me), orders, admin, erp (connectors)
│     └─ services/
│        ├─ erpClient.js      Mini-ERP client: REST + OData, basic/OAuth, timeout, retry
│        ├─ connectors.js     SAP connector table (add, edit, delete, active)
│        ├─ serviceKey.js     XSUAA credentials: key file or CF binding
│        ├─ productSync.js    ERP -> shop: products and order statuses
│        ├─ erpSync.js        when to sync: start, every 60 s, "Sync now", connector switch
│        ├─ erpOrders.js      shop -> ERP: customer and order (ERP first)
│        ├─ erpMasterData.js  shop -> ERP: add/edit products, load the demo catalog
│        └─ inventory.js      stock reservation without an ERP
└─ client/src/
   ├─ main.jsx, App.jsx       entry point, page routing, header/footer
   ├─ api.js                  all /api calls, money/date formatting
   ├─ state.jsx               session (Auth) and cart (Cart) contexts
   ├─ pages/                  Shop, Product, Cart, Checkout, Confirmation,
   │                          Login, Register, Account, Admin
   └─ components/             SapConnectors, OrderCard, ProductArt, Modal, Toast ...
```

Every line of the code has an English comment above it – in the presentation you can open a file and explain from there.

Address (screen)	Page	What does it do?
/	ShopPage	Product list, categories, search, sorting, “in stock only”, add to cart.
/products/:id	ProductPage	Product details, quantity, stock information.
/cart	CartPage	Cart: change quantities, remove, subtotal.
/checkout	CheckoutPage	Delivery address + order summary, “Place order”.
/order-confirmation/:id	ConfirmationPage	Confirmation; ERP order number (SAP ERP #…).
/login, /register	LoginPage, RegisterPage	Login (with demo account buttons), registration.
/account	AccountPage	Profile and address, purchase history, password change.
/admin	AdminPage	Tabs: Overview, Products, Orders, Customers, SAP connectors (admins only).

5. Data model (SQLite)

The tables are defined in `server/src/db.js`. Design principle: **stay close to the ERP’s model** (the same product key productID, the same address fields, the same order status codes) – so the integration needs almost no conversion.

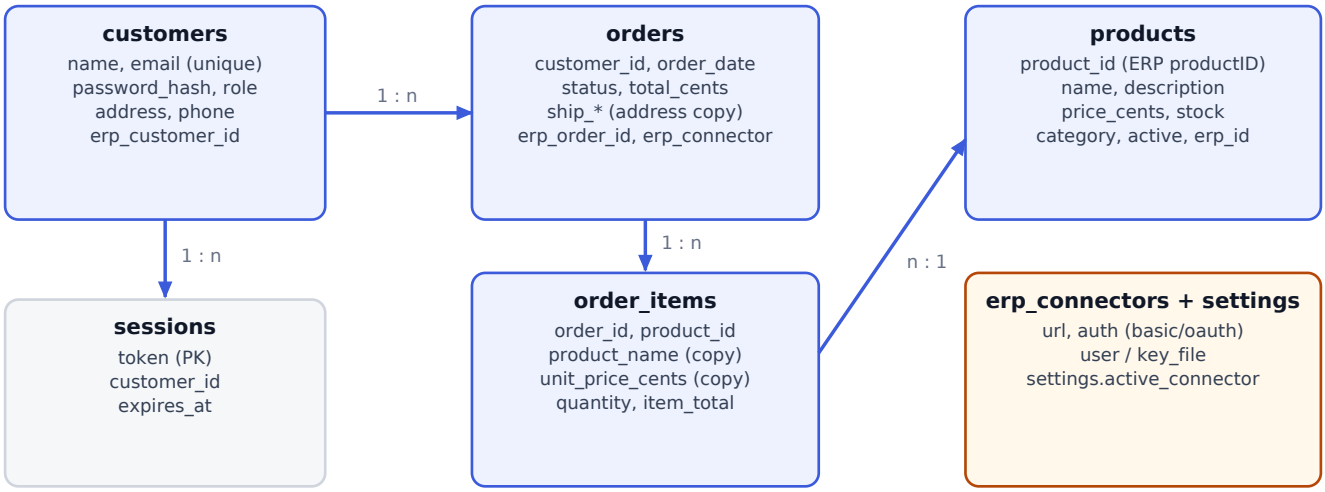


Figure 2 – Tables and relations. ERP keys (erp_*) are stored together with the connector they belong to.

Table	Important fields	Design decision
products	`product_id` (business key = ERP productID), name, description, `price_cents`, currency, stock (0–1000), category, `active`, `erp_id`, `erp_connector`, `erp_created_by`	Money in **cents (integer)**: no decimal rounding errors. **active=0** instead of deleting (soft delete): old orders stay intact. The category belongs to the shop only.
customers	name, `email` (unique, case-insensitive), `password_hash`, address fields (same as the ERP), phone, role, `erp_customer_id`	Passwords are never stored as plain text (bcrypt).
sessions	token, customer_id, expires_at	Server-side sessions: invalid after logout or expiry.

Table	Important fields	Design decision
orders	customer_id, order_date, **status**, **total_cents**, ship_* address copy, `erp_order_id`, `erp_order_number`, `erp_connector`	The address is copied at order time; the order stays correct even if the customer changes the address later.
order_items	product_id, **quantity**, product_name and unit_price_cents (snapshot), item_total_cents	“Products purchased + quantities”. The history stays correct even if the price changes later.
erp_connectors, settings	id, name, url, auth, username/password or key_file, auto_catalog; `active_connector`	Which ERP we are connected to is stored in the database and can be changed in the UI.

Shop status	ERP OrderStatus	In the lab text	ERP action
NEW	NEW	pending	— (when the order is created)
PICKED	PICKED	(being prepared)	`pickOrder`
SHIPPED	SHIPPED	shipped	`shipOrder`
COMPLETED	COMPLETED	completed	`completeOrder`
CANCELED	CANCELED	(canceled)	`cancelOrder` (known ERP bug: always 422)

6. Product management (R1)

- **List:** `GET /api/products` returns the products of the active connector (or the shop’s own products without a connector). If the list is empty and the ERP has just started, the request tries a synchronization (waits at most 3 s). Interface: category chips, search, sorting, stock badges like “Only 3 left”.
- **Add (admin):** `POST /api/products` → validation (ID format, price ≥ 0, stock 0–1000) → with an ERP connected, **the ERP first** with `OData POST /Products` → then synchronized from the ERP. If the ERP refuses, no product is created in the shop either.
- **Edit (admin):** `PUT /api/products/:id` → `PATCH /Products(<uuid>)` in the ERP → synchronization. The category and “listed in the shop” exist only in the shop.
- **Remove (admin):** `DELETE /api/products/:id` → **soft delete** (active=0). The product stays in the ERP (other systems may use it); the admin can undo it with “Restore”.
- **Source:** Admin › Products shows for every product “ERP initial data” (the lecture data, e.g. MTB/Gravel) or “added via API” (added to the ERP by the shop).

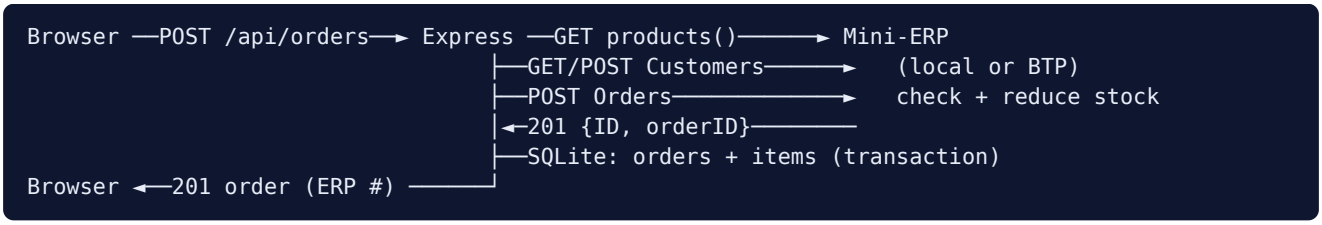
7. Customer management (R2)

- **Registration:** `POST /api/auth/register` – name, email (format + uniqueness), password (at least 6 characters), address. The password is hashed with **bcrypt**. With an ERP connected, the customer is created in the ERP too (or at the first order if the ERP is down).
- **Login:** `POST /api/auth/login` → random session token (sessions table). The browser sends `Authorization: Bearer <token>` with every request. A wrong email and a wrong password give the same message (nothing is revealed).
- **Profile:** `GET/PUT /api/me` – name, email, phone, address; a password change needs the current password. The change is written to the ERP customer too with `PATCH /Customers`.
- **Purchase history:** My account › Orders → `GET /api/orders` (own orders only; another customer's order returns 404).
- **Roles:** customer and admin. Admin endpoints are protected with `requireAdmin` (401 not logged in, 403 not allowed).
- **Demo accounts:** one click on the login page (admin + customer "Max Mustermann"). The values are in `server/.env.example`; in the cloud the admin account is not shown on the login page, its password is set with `cf set-env`.

8. Order flow (R3): "ERP first"

The most important design decision: **the order is created in the ERP first, then in the shop.** The ERP checks the stock. If the ERP does not accept the order (not enough stock, ERP down), no order is created in the shop either – the two systems never contradict each other.

Step	What happens?	Explanation
1	**Cart**	The customer adds products to the cart (in the browser, localStorage).
2	**POST /api/orders**	The server merges the lines, checks the quantities and reads the price **from the database**.
3	**Product sync**	The products are read from the active ERP: current ERP keys and prices.
4	**ERP customer**	Looked up in the ERP by email, or created with `POST /Customers`.
5	**ERP order**	`POST /Orders` (customer, items, amounts). The ERP checks and reduces the stock; if there is not enough, **409 "exceeds stock"**. This call is **never retried automatically** (risk of a duplicate order).
6	**SQLite transaction**	orders + order_items are written and the local stock is reduced at once (with the price/total from the ERP). Either all of it happens or nothing.
7	**Response**	The confirmation page shows the "SAP ERP #…" number; the stock is refreshed from the ERP in the background.



Checkout sequence (simplified). Code: routes/orders.js and services/erpOrders.js.

Status changes

- Status buttons in Admin › Orders → `PATCH /api/orders/:id/status` → the matching action in **the ERP where the order was created** (pickOrder, shipOrder, completeOrder). If the ERP accepts, the shop is updated.
- Changes made in the ERP (in the Fiori app) are pulled into the shop every 60 seconds (`syncOrderStatusesFromErp`).
- A customer can cancel an own order in status NEW; however, the Mini-ERP's `cancelOrder` action has a bug (always 422) – in that case the shop does not change the order (section 13).
- Without an ERP (standalone mode) the shop manages the stock itself: a conditional `UPDATE ... WHERE stock >= ?` makes sure the last item cannot be sold twice.

9. SAP ERP integration and SAP connectors

Data	Master (source of truth)	How does the shop read / write it?
Products, prices, stock	**SAP ERP**	Read: REST `/rest/api/products()` (at start, every 60 s, "Sync now", before checkout). Write: OData POST/PATCH `/Products`.
Orders	**SAP ERP**	OData `POST /Orders` (checkout); actions pick/ship/complete; statuses are synchronized back.
Customers	Shop + ERP	Found by email / `POST /Customers`; profile changes with `PATCH`.
Passwords, sessions, cart, categories	Shop	Only SQLite / the browser.

SAP connectors (Admin › SAP connectors)

Which ERP we connect to is not fixed in the code; it is stored in the `erp_connectors` table and managed in the admin interface. **One** connector is active at a time (or none).

Connector	Address	Authentication	Property
Local SAP (cds watch)	http://localhost:4004	Basic auth: service-user (the ERP's mocked user)	Default. "Demo catalog: created automatically" – loads the 14 products again when the ERP restarts.
SAP BTP (Cloud Foundry)	…-simple-erp-srv.cfapps.us10-001…	OAuth 2.0 client credentials (XSUAA). Locally a key file (`xsuaa-key.json`), in the cloud a binding (`vcap:simple-erp-auth-api`).	Data in HANA, persistent.

- **Buttons on the cards:** Test (REST + OData check, time and number of products), Use this ERP (activate + synchronize at once), Edit, Delete (not possible for the active connector or one with orders), Add connector (ready-made Local/BTP templates, tested before saving).
- **Why are the keys per connector?** The same product (DXTR1000) has different UUIDs in the local ERP and on BTP. That is why `erp_connector` is stored together with `erp_id`; products are matched by the business key `productID`.

- **Fault tolerance:** a 10 s timeout on every ERP call; GET (and PATCH) requests are retried twice on network errors/5xx (waiting 0.5 s and 1 s = exponential backoff); **POST is never retried** (not idempotent); the OAuth token is cached and renewed once on a 401; business errors of the ERP (409 stock) are passed on to the user.
- **ERP down:** instead of showing old products, the shop says “catalog not available”; the admin sees the error and what has to be started.

10. Running it: local and cloud

Local

```
# One click: webshop\start-local.cmd  (ERP + shop + browser)
# or by hand, in two terminals:
cd C:\Users\ONUR\systems-integration\src\simple-erp
npx.cmd cds watch                       # Mini-ERP  -> http://localhost:4004
cd C:\Users\ONUR\Desktop\webshop
npm.cmd run dev                         # API :3001 + React :5173 -> http://localhost:5173
```

Cloud (SAP BTP Cloud Foundry)

```
cd C:\Users\ONUR\Desktop\webshop
npm.cmd run cf:build                    # React build + cf-dist/ (code + manifest.yml)
cd cf-dist
cf push                                 # app "velocity-webshop", binding simple-erp-auth-api
cf set-env velocity-webshop ADMIN_PASSWORD 'YourPassword'
cf restart velocity-webshop
```

Before a presentation: start HANA → cf start simple-erp-srv → cf start velocity-webshop (Lab 0 guide, section 9).

	Local shop	Cloud shop
Address	http://localhost:5173	https://369eca91trial-dev-velocity-webshop.cfapps.us10-001.hana.ondemand.com
Default ERP	Local SAP (cds watch)	SAP BTP (the only connector)
ERP identity	basic auth (service-user)	OAuth, service binding (VCAP_SERVICES)
Shop data	server/data/webshop.db (persistent)	container disk: reset on restart/push
ERP data	in memory, reset with cds watch	persistent in HANA

Two windows side by side in the presentation

Left: the local shop (Local SAP) – right: the cloud shop (SAP BTP). Place an order in each and show that the stock goes down in the matching ERP (the Fiori app on localhost:4004 / the ERP on BTP). Switch the connector live in Admin › SAP connectors.

11. Design decisions (to defend)

#	Decision	Why?	Alternative / cost
1	The data model is close to the ERP (productID, address fields, status codes)	No conversion in the integration, less risk of errors.	If the ERP changes, the model is affected too.
2	ERP = master; synchronized copy in the shop	One source of truth; the shop is fast and can still read during a short ERP outage.	The copy can be up to ~60 s old.
3	“ERP first” checkout	The ERP decides about the stock; the shop and the ERP never contradict each other.	No orders while the ERP is down.
4	The order POST is never retried automatically; reads are retried with timeout + backoff	A retry after a timeout could create the same order twice (not idempotent).	The user retries after an error.
5	order + order_items with a snapshot of price and name; money in cents (integer)	Old orders stay correct even if prices change; no rounding errors.	Some duplicated data.
6	Checkout in one DB transaction; conditional stock UPDATE without an ERP	No half-written orders; the last item cannot be sold twice.	—
7	Deleting a product = soft delete	The order history stays intact; the product stays in the ERP and can be restored.	The table grows.
8	Server-side sessions + bcrypt; role checks on the server	Sessions can be revoked; leaked hashes cannot be reversed.	One DB query per request.
9	SQLite	No installation; everybody in the team can run it; the relational model fits.	Not persistent in the cloud (section 13).
10	One ERP client, two authentication methods (basic / OAuth)	The same code works with the local and the BTP ERP.	—
11	SAP connectors (switch the ERP at runtime)	Local ↔ BTP with one click; keys per connector.	An extra table and more complexity.
12	No secrets in the code/DB (key file, binding, .env; .gitignore)	Security; nothing leaks in a zip/commit.	One more setup step.

12. Questions the lecturer may ask

Q: If the ERP is the master anyway, why do you keep the products in the shop as well?

A: Speed and resilience: going to the ERP for every page view would be slow, and during a short ERP outage the shop can still show the last known data. The copy is refreshed every 60 s and before every checkout; the decision (stock) is still made by the ERP.

Q: What happens if the ERP is down during checkout?

A: The ERP request fails → 503 “ERP not reachable”, no order is created in the shop and the customer sees an error message. No inconsistent data is created.

Q: What if two people buy the last bike at the same time?

A: The ERP checks and reduces the stock; the second order gets 409 “exceeds stock” and the shop shows it to the user. Without an ERP, the conditional UPDATE gives the same protection.

Q: Why don’t you retry the order request?

A: POST is not idempotent: if the request reached the ERP but the response was lost, a retry would create a second order. Read requests (GET) are retried safely.

Q: How does the shop authenticate with the ERP?

A: Locally with the ERP’s mocked user (basic auth, role system-integration). On BTP with OAuth 2.0 client credentials: a token is obtained from XSUAA with the identity of simple-erp-auth-api; the token contains the authority system-integration (xs-security.json: grant-as-authority-to-apps).

Q: Can a user change the price?

A: No. The browser only sends the product ID and the quantity; the price is read on the server from the database (the ERP price).

Q: What happens to old orders when a product is deleted?

A: Soft delete: the product row stays (active=0); the order items already contain a copy of name and price.

Q: What about old orders when a customer changes the address?

A: The order contains a copy of the delivery address (ship_*); old orders do not change.

Q: Can the shop and the ERP become inconsistent?

A: One open point: if the ERP order was created but the local record fails, the ERP order is not rolled back (in ERP mode the local stock update cannot fail, so the risk is very low). Idea for a fix: periodic reconciliation by erp_order_id / an outbox.

Q: How do the labs after Lab 1 affect this design?

A: The integration layer is separate (services/): file transfer (Lab 2) or another RPC/messaging approach (Labs 3–5) can be added as a new service/connector; the routes and the interface stay the same.

13. Known limitations

Topic	Explanation	What could be done?
ERP cancelOrder bug	In the Mini-ERP, `srv/simpleerp.js` compares the selected row object with the text "NEW" → cancelling always returns 422.	Must be fixed in the ERP; the shop behaves correctly (changes nothing).
SQLite in the cloud	The container disk is temporary: after restart/push the shop users and the shop's order list are reset.	For persistence: a HANA HDI container or a PostgreSQL service.
Trial stops at night	HANA and the applications stop every night.	Start order before a presentation (Lab 0 guide).
Work Zone	Subscription failed (needs an IAS trust).	For the Fiori launchpad; not needed for the shop/API.
ERP order + local record	The inconsistency scenario above.	Reconciliation job / outbox pattern.

14. Glossary (short)

Term	Meaning
SPA	Single Page Application: a React application that works without reloading the page.
REST API	Access to resources with HTTP + JSON (GET/POST/PUT/PATCH/DELETE).
OData v4	SAP's standard REST-based data protocol ($filter, actions).
System of record / master	The system where a piece of data is "really" kept (for us: the ERP).
Idempotent	A request that gives the same result when it is repeated (GET yes, the order POST no).
Exponential backoff	Increasing the waiting time with every retry (0.5 s → 1 s).
OAuth 2.0 client credentials	An application gets a token with its own identity (client id/secret).
Service binding / VCAP_SERVICES	Cloud Foundry passes the data of a service to the application automatically.
Soft delete	Marking a row as inactive instead of deleting it.
Transaction	Several database operations that happen either all together or not at all.